

Interactive Sales Dashboard: Project Report

1. Project Overview

This project delivers an interactive sales dashboard integrating static statistical analysis (via Seaborn) and dynamic, interactive data exploration (via Plotly). The objective is to provide comprehensive insights into sales trends, customer segmentation, and product performance within a unified web interface, maintaining a modern, high-contrast visual aesthetic.

2. Setup Instructions

Follow these steps to configure the environment and run the dashboard:

- Ensure Python 3.9+ is installed on your system.
- Open a terminal and navigate to the project directory.
- Create a virtual environment: `python -m venv venv`
- Activate the environment: `source venv/bin/activate` (Linux/Mac) or `venv\Scripts\activate` (Windows)
- Install dependencies: `pip install -r requirements.txt`
- Launch the dashboard: `streamlit run dashboard.py`

3. Code Structure

The repository is organized as follows to separate analysis, deployment, and assets:

- `dashboard.py`: Main Streamlit application file combining all visualizations into a web layout.
- `dashboard.ipynb`: Jupyter notebook containing the Day 1-5 exploratory data analysis and individual chart development.
- `visualizations/`: Directory for saving static output (e.g., Seaborn plots saved as PNGs or PDFs).
- `requirements.txt`: Lists all Python packages required for execution.
- `dashboard_demo.gif`: Visual demonstration of the interactive features.

4. Dashboard Guide & Visualization Interpretation

Seaborn Visualizations (Statistical)

1. Price Distribution by Category (Boxplot): Identifies outliers and the interquartile range of pricing across product lines.
2. Customer Age vs Sales (Violin Plot): Showcases the probability density of sales volume across different demographic age groups.
3. Feature Correlation (Heatmap): Uncovers the statistical relationship between numerical variables (e.g., discount rate vs. sales volume).

Plotly Visualizations (Interactive)

4. Revenue Over Time (Line Chart): Features hover-over data points and zoom capabilities to isolate specific fiscal quarters.
5. Product Performance (Sunburst/Treemap): Allows users to click and drill down into sub-categories to see granular revenue share.

5. Technical Details & Architecture

The architecture utilizes Streamlit as the bridging framework. Standard Python scripts cannot render HTML/JS Plotly components alongside Matplotlib/Seaborn canvas elements natively in a cohesive window. Streamlit resolves this by serving a local web page. Data manipulation is handled by Pandas, ensuring $O(N)$ time complexity for most aggregations and group-by operations before passing the reduced datasets to the rendering libraries.

6. Testing Evidence

Validation strategies implemented:

1. Data Quality: Asserts are used to check for null values in critical columns (e.g., Price, Category).
2. Component Rendering: Streamlit's testing framework verifies that all 5 chart components load without exceptions.
3. Responsiveness: Manual testing confirming UI scales correctly on 1080p and 1440p displays.